Conteúdos complementares de Banco de Dados

Sistemas de Banco de Dados I

Esta apresentação complementa os conteúdos sobre usuários de banco de dados, linguagens SQL, entidades fracas e critérios de uso de SGBD. O objetivo é estudar conceitos que aparecem na prática profissional e que ajudam a interpretar melhor arquiteturas, modelos e decisões de projeto.

Usuários

SQL DDL/DML/DCL

Entidade fraca

Quando NÃO usar SGBD

Apoio de estudo conteúdos complementares

Materiais-base: slides 01 a 06 de Banco de Dados I

Objetivos da aula

O QUE O ESTUDANTE DEVERÁ COMPREENDER

Ao final, o estudante deverá diferenciar usuários casuais e sofisticados, reconhecer DDL, DML e DCL, explicar chave parcial em entidade fraca e avaliar quando um SGBD pode ser desnecessário. Esses tópicos se conectam aos slides de introdução, SGBDs, arquitetura e modelagem.

Usuários

SQL
DDL/DML/DCL

Entidade fraca

Quando NÃO usar SGBD

Apoio de estudo
conteúdos complementares

Revisar: Slide 01, slides 2, 10 e 11; Slide 02, slides 10 a 16; Slide 05, slides de modelagem.

Por que estes tópicos importam?

DA TEORIA À TOMADA DE DECISÃO

Banco de dados não é apenas uma tabela. Envolve pessoas, comandos, regras e escolhas técnicas. Saber quem usa o sistema, que comando altera estrutura, que entidade depende de outra e quando um SGBD é exagerado evita erros comuns em projetos acadêmicos e profissionais.

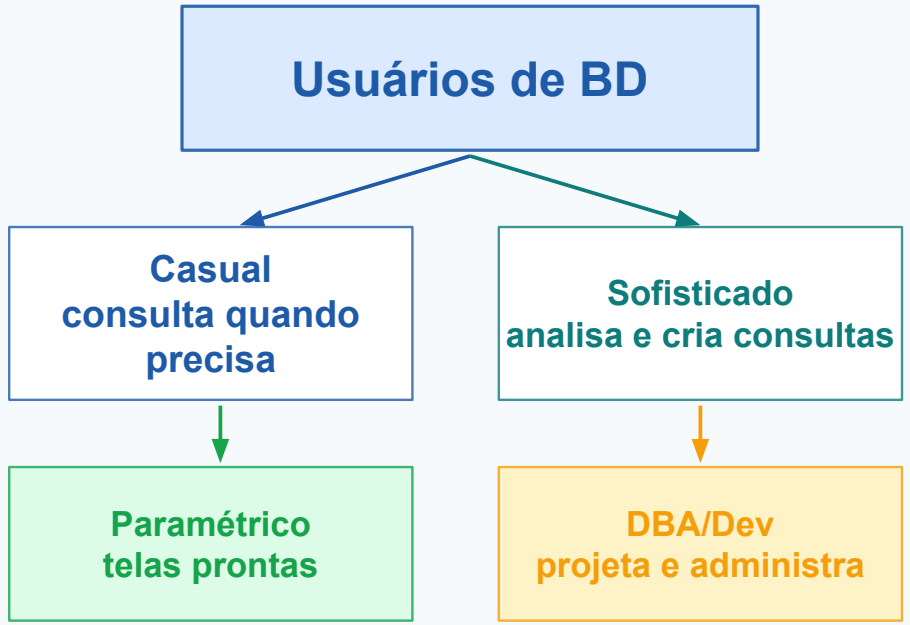


Revisar: Slide 01, slides 2 e 10; Slide 02, slide 1; Slide 03, slides 2 a 8.

Usuários de banco de dados

PERFIS DE INTERAÇÃO COM O SGBD

Um sistema de banco de dados atende pessoas com necessidades diferentes. Alguns usuários apenas consultam dados por telas prontas; outros escrevem consultas, cruzam informações e produzem análises. Entender esses perfis ajuda a projetar interfaces, permissões, relatórios e níveis de acesso adequados.

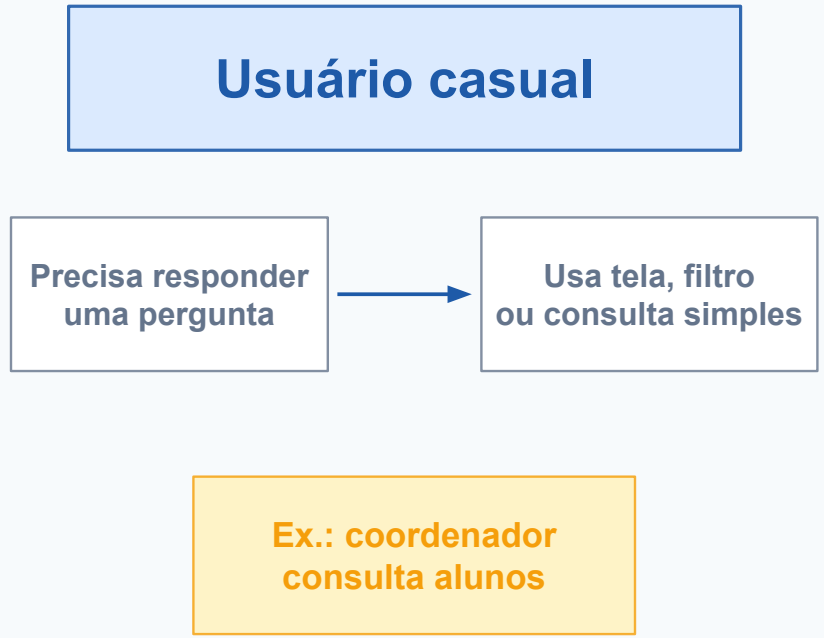


Complementa: Slide 02, componentes de SGBD e arquitetura cliente/servidor.

Usuário final casual

CONSULTAS EVENTUAIS E PONTUAIS

O usuário final casual acessa o banco quando precisa responder a uma dúvida específica. Ele normalmente usa telas, filtros, relatórios ou consultas simples preparadas por outra pessoa. Seu foco é obter uma informação confiável, sem conhecer detalhes internos do banco de dados.

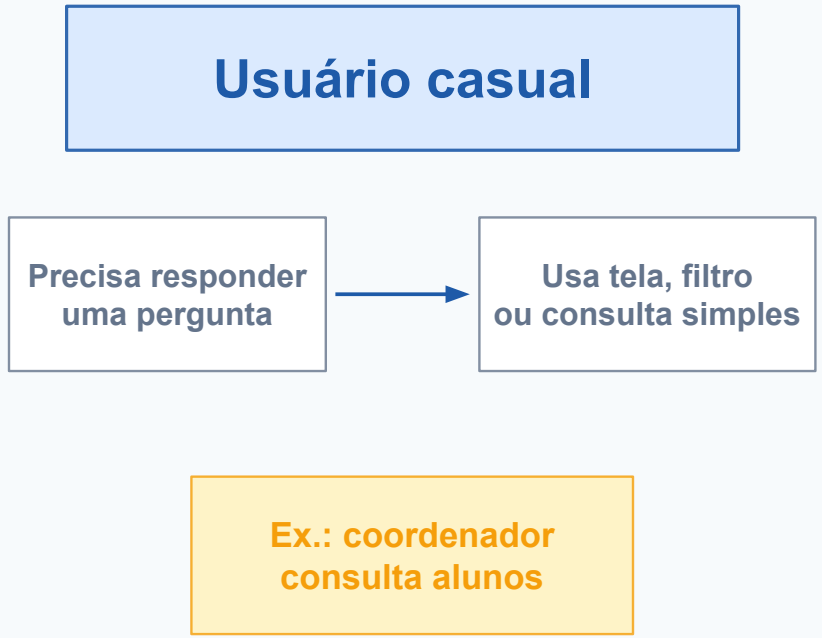


Exemplo: coordenador consulta matrícula, situação acadêmica ou histórico de um estudante.

Exemplo de usuário casual

CONSULTA EM UM SISTEMA ACADÊMICO

Imagine uma secretaria acadêmica verificando se um aluno está matriculado em determinada disciplina. A pessoa informa nome ou matrícula, aplica um filtro e visualiza o resultado. Ela não precisa saber como as tabelas estão relacionadas, mas depende de dados íntegros e atualizados.

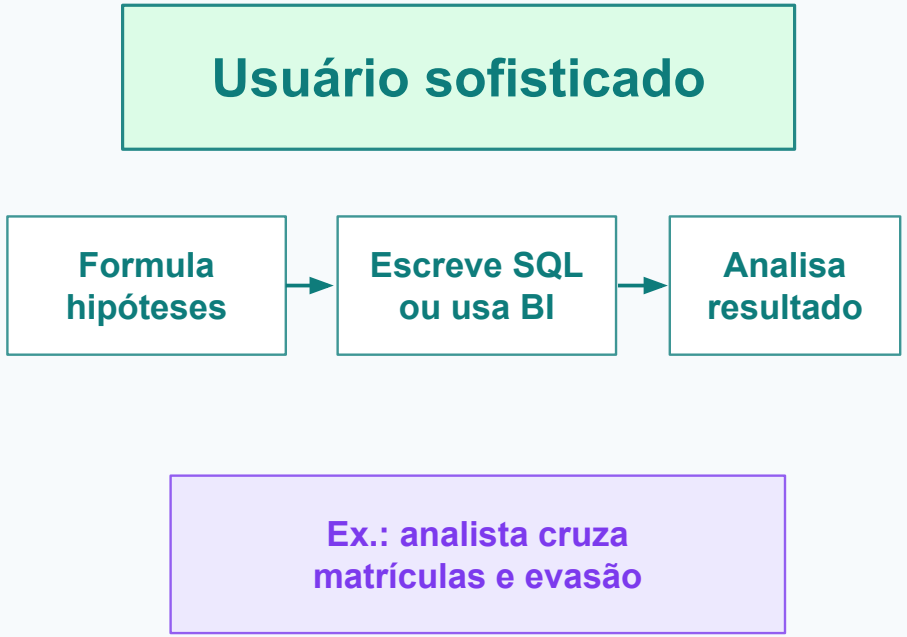


Demonstração sugerida: usar uma tela simulada ou filtro em planilha para mostrar a ideia.

Usuário final sofisticado

CONSULTAS FLEXÍVEIS E ANÁLISES

O usuário sofisticado compreende melhor os dados e costuma formular consultas próprias. Pode usar SQL, ferramentas de BI ou relatórios avançados para combinar informações. Ele não apenas procura um registro: investiga padrões, compara resultados e transforma dados em apoio à decisão.

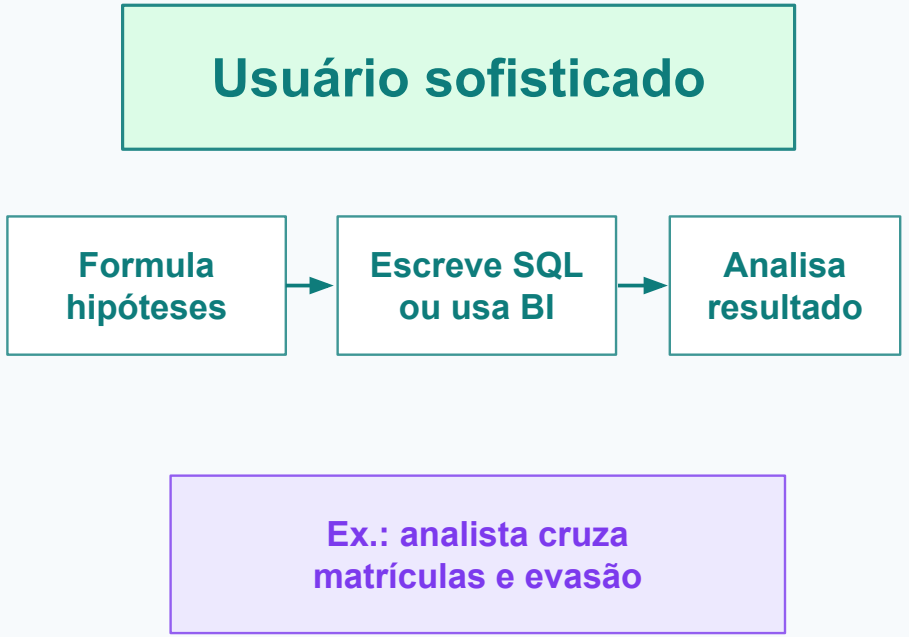


Exemplo: analista compara evasão por curso, período e turno.

Exemplo de usuário sofisticado

ANÁLISE COM SQL OU BI

Em vez de perguntar apenas se um aluno está matriculado, o usuário sofisticado pode investigar quantos alunos desistiram por semestre, quais cursos concentram maior evasão e quais turnos apresentam maior reprovação. Para isso, combina tabelas, filtros, agrupamentos e interpreta os resultados.

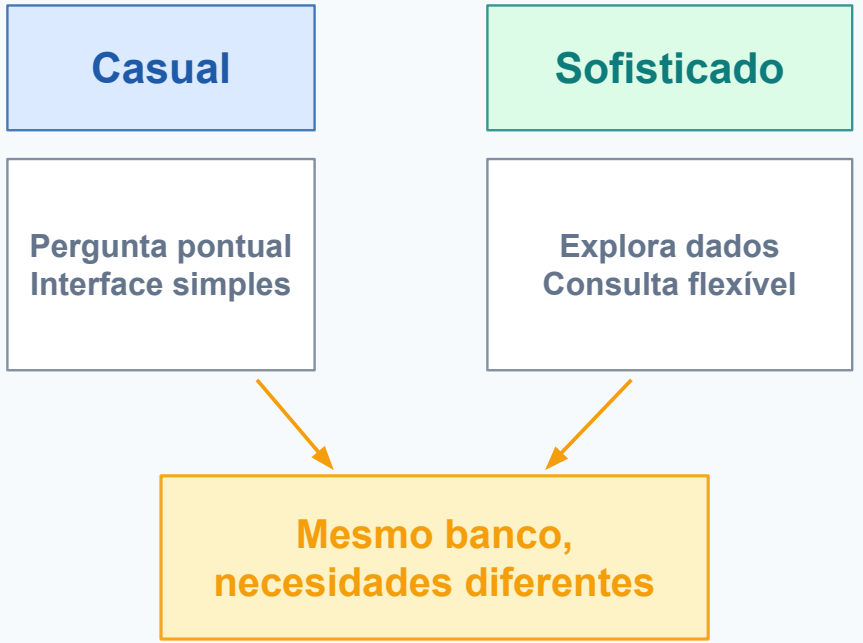


Ferramentas gratuitas: DB Browser for SQLite, SQLiteOnline, Google Looker Studio ou Metabase Community.

Casual x sofisticado

COMPARAÇÃO DIDÁTICA

A diferença central está na liberdade de consulta e no grau de conhecimento sobre os dados. O usuário casual usa respostas mais prontas e seguras. O sofisticado explora hipóteses, combina tabelas e interpreta resultados. Ambos precisam de dados consistentes e permissões compatíveis.

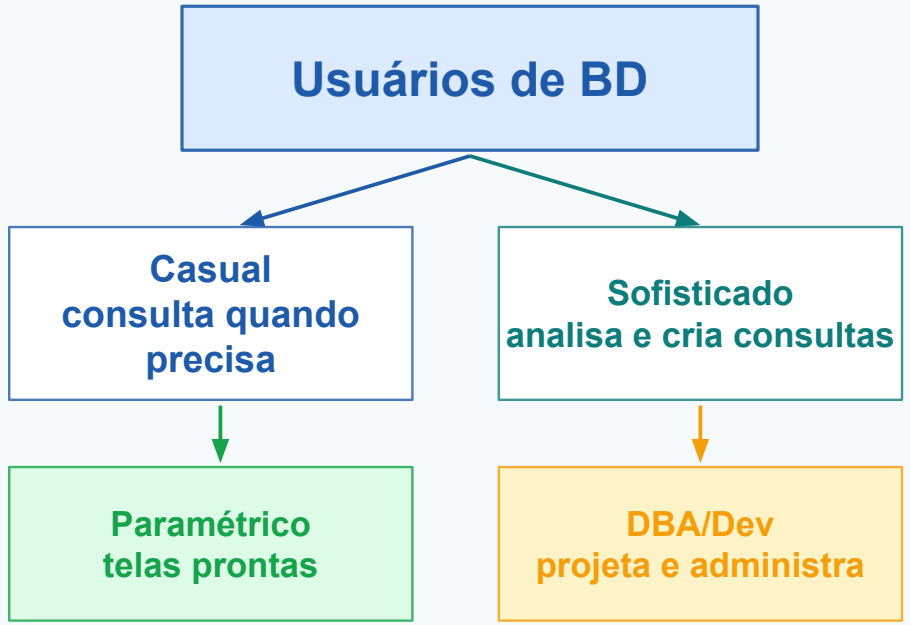


Conteúdo complementar ao estudo de SGBDs e segurança de acesso.

Outros perfis importantes

CONTEXTO PARA NÃO CONFUNDIR

Além dos usuários casuais e sofisticados, existem usuários paramétricos, que executam operações repetitivas em telas padronizadas, desenvolvedores de aplicação, que criam sistemas, e administradores de banco de dados, que cuidam de segurança, desempenho, backup e configuração do ambiente.

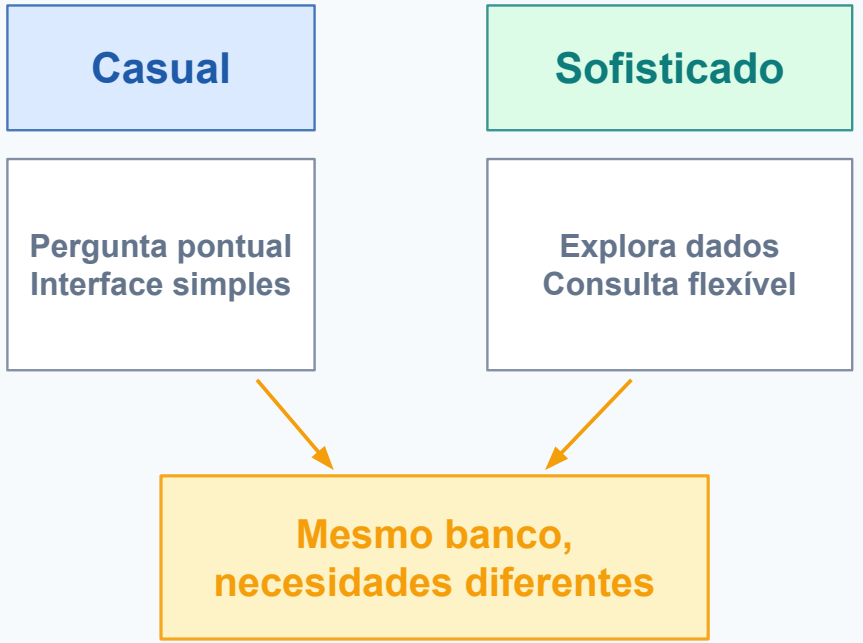


Revisar: Slide 02, componentes do SGBD, arquitetura e segurança.

Estudo de caso: biblioteca

MESMO BANCO, VÁRIOS USUÁRIOS

Em uma biblioteca, o atendente registra empréstimos em telas prontas, o coordenador consulta relatórios ocasionais, o analista verifica livros mais emprestados e o administrador define permissões. Todos usam o mesmo banco, mas cada perfil precisa de acesso, interface e responsabilidade diferentes.



Atividade rápida: listar quatro usuários de um sistema de biblioteca e suas ações.

Prática: mapear usuários

PASSO A PASSO DE ANÁLISE

Para mapear usuários de um sistema, liste quem interage com os dados, descreva o que cada um precisa fazer, classifique se a ação é consulta, cadastro, alteração ou administração e defina o nível mínimo de permissão. Esse processo reduz riscos e melhora o projeto.

Preciso de SGBD?

- **Muitos usuários?**
- **Dados críticos?**
- **Consultas variadas?**
- **Segurança?**

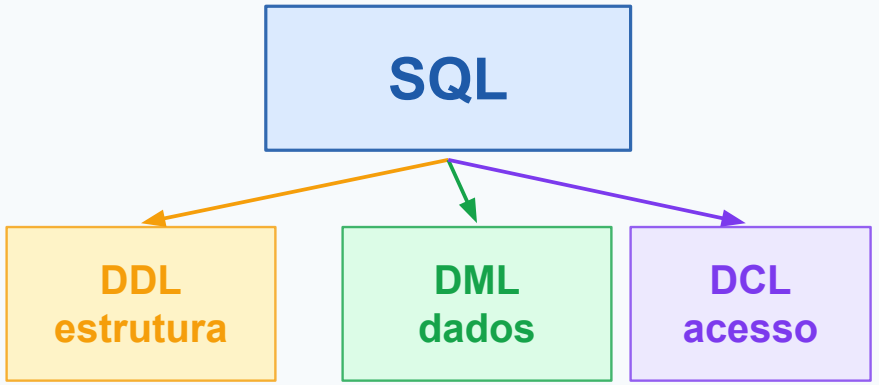
Se várias respostas forem “sim”, o SGBD tende a fazer sentido.

Aplicação gratuita sugerida: diagrams.net para desenhar perfis e permissões.

Famílias da linguagem SQL

DDL, DML E DCL

SQL é frequentemente organizado por finalidade. DDL define estruturas, DML consulta e manipula dados, e DCL controla privilégios. Essa divisão é didática: ajuda a entender se um comando muda o desenho do banco, as linhas armazenadas ou as permissões de acesso.



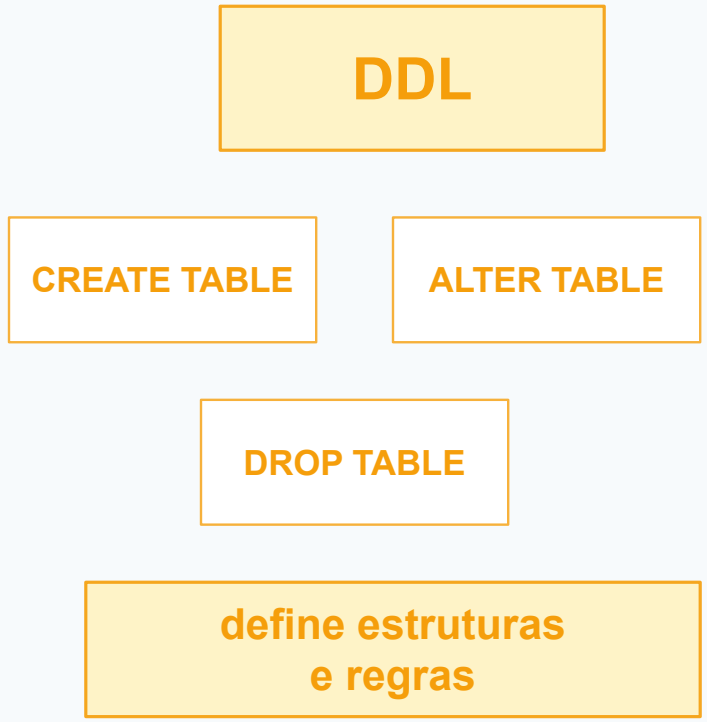
Organiza comandos pela finalidade

Revisar: Slide 04, slides sobre DDL e criação de tabelas. Complementar: DML e DCL.

DDL: Data Definition Language

COMANDOS DE DEFINIÇÃO DE DADOS

DDL é usada para criar, alterar ou remover estruturas do banco. Quando criamos uma tabela, definimos colunas, tipos, chaves e restrições. Portanto, DDL altera o esquema, isto é, a estrutura que organiza os dados antes de eles serem inseridos.



Exemplos: CREATE TABLE, ALTER TABLE, DROP TABLE. Revisar: Slide 04, slides 31 a 34.

DML: Data Manipulation Language

COMANDOS PARA LER E ALTERAR DADOS

DML trabalha com os registros armazenados nas tabelas. Ela permite consultar, inserir, atualizar e excluir linhas. Diferente da DDL, a DML não muda a estrutura da tabela; ela muda ou lê o conteúdo existente em determinado momento.

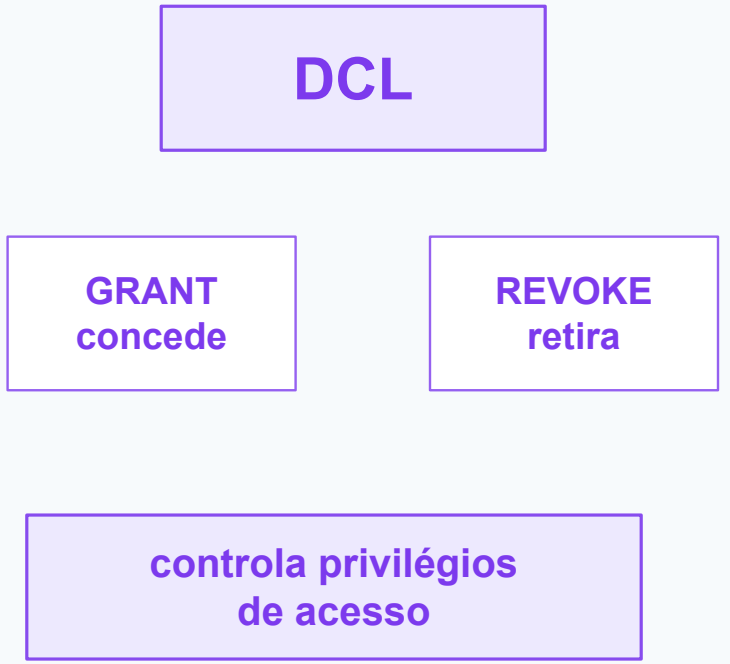


Exemplos: SELECT, INSERT, UPDATE, DELETE.

DCL: Data Control Language

COMANDOS DE CONTROLE DE ACESSO

DCL é usada para conceder ou retirar permissões. Em sistemas reais, nem todo usuário deve acessar tudo. Um perfil pode apenas consultar, outro pode inserir registros e outro pode administrar tabelas. DCL ajuda a aplicar o princípio do menor privilégio.

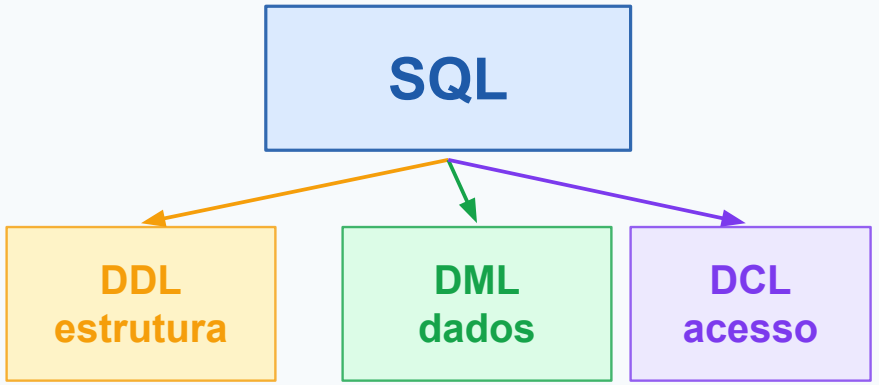


Exemplos: GRANT e REVOKE. Complementa os conteúdos de segurança e usuários.

Comparando DDL, DML e DCL

QUAL É A FINALIDADE DO COMANDO?

Uma forma simples de diferenciar as famílias é perguntar: o comando muda a estrutura? Então é DDL. O comando lê ou altera registros? Então é DML. O comando concede ou retira acesso? Então é DCL. Essa pergunta evita confusão entre comandos parecidos.



Organiza comandos pela finalidade

Dica de estudo: classifique comandos por finalidade, não apenas por memorização.

Prática: ambiente para SQL

FERRAMENTAS GRATUITAS PARA TREINO

Para demonstrar DDL e DML em aula, use DB Browser for SQLite ou SQLiteOnline. São opções simples para criar tabelas, inserir registros e executar SELECT sem instalar servidores complexos. Para DCL, prefira PostgreSQL local, pgAdmin ou um ambiente acadêmico controlado.

1. Abrir DB Browser
2. Criar tabela ALUNO
3. Inserir registros
4. Consultar com SELECT
5. Interpretar erros

DB Browser for SQLite: https://sqlitebrowser.org/ | SQLiteOnline: https://sqliteonline.com/

Passo a passo: DDL e DML

EXEMPLO COM TABELA ALUNO

Abra o DB Browser for SQLite, crie um novo banco, execute CREATE TABLE ALUNO com id, nome e curso, insira duas linhas com INSERT e consulte com SELECT. Depois altere um nome com UPDATE e observe que a estrutura continua a mesma.

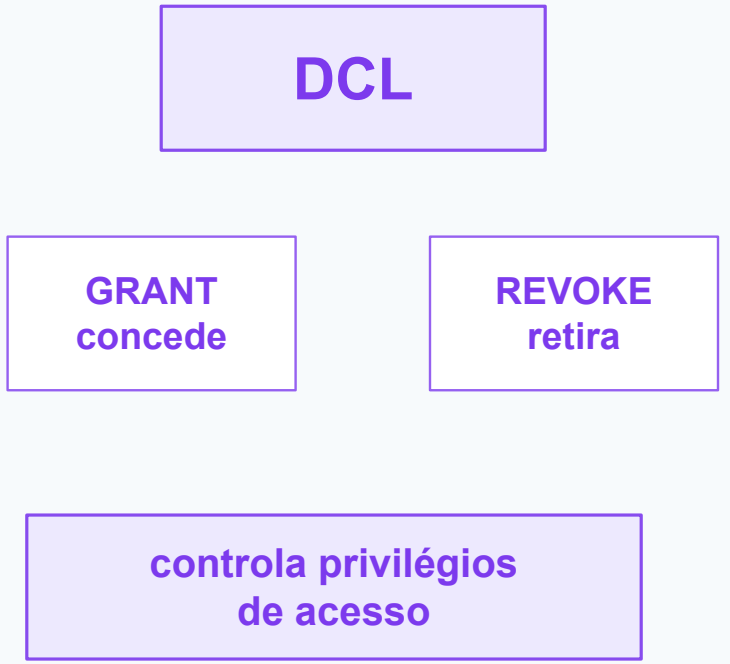
1. Abrir DB Browser
2. Criar tabela ALUNO
3. Inserir registros
4. Consultar com SELECT
5. Interpretar erros

Use uma tabela pequena para focar no conceito, não na complexidade do script.

Passo a passo: DCL

EXEMPLO CONCEITUAL COM POSTGRESQL

Em PostgreSQL, o professor pode criar um usuário leitor, conceder SELECT em uma tabela e depois revogar a permissão. O objetivo didático é mostrar que segurança não depende apenas de senha: depende também de privilégios sobre objetos do banco.



Exemplo: GRANT SELECT ON TABLE aluno TO leitor; REVOKE SELECT ON TABLE aluno FROM leitor.

Entidade forte

EXISTÊNCIA INDEPENDENTE

Uma entidade forte possui identificação própria e não depende de outra entidade para existir no modelo. Cliente, Pedido ou Produto podem ser fortes quando possuem uma chave que identifica cada ocorrência. Em diagramas ER, elas costumam aparecer como entidades normais.



Revisar: Slide 05, slides sobre entidades fortes e fracas; Slide 06, elementos de DER.

Entidade fraca

DEPENDÊNCIA DE IDENTIFICAÇÃO

Uma entidade fraca depende de uma entidade forte para ser identificada de forma completa. Ela pode até ter um identificador local, mas esse valor só faz sentido dentro do contexto da entidade proprietária. Por isso, costuma aparecer ligada a um relacionamento identificador.

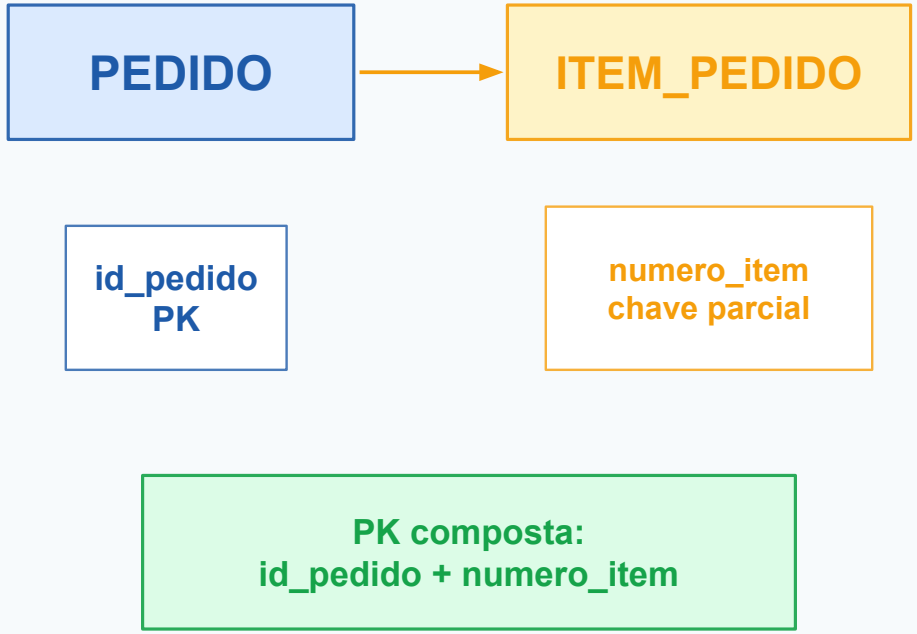


Exemplo: ItemPedido depende de Pedido; Dependente pode depender de Funcionário.

Chave parcial

O IDENTIFICADOR LOCAL DA ENTIDADE FRACA

A chave parcial distingue ocorrências da entidade fraca apenas dentro da entidade forte relacionada. No exemplo ItemPedido, numero_item diferencia os itens dentro de um pedido. Para identificar totalmente o item, combinamos id_pedido com numero_item em uma chave composta.

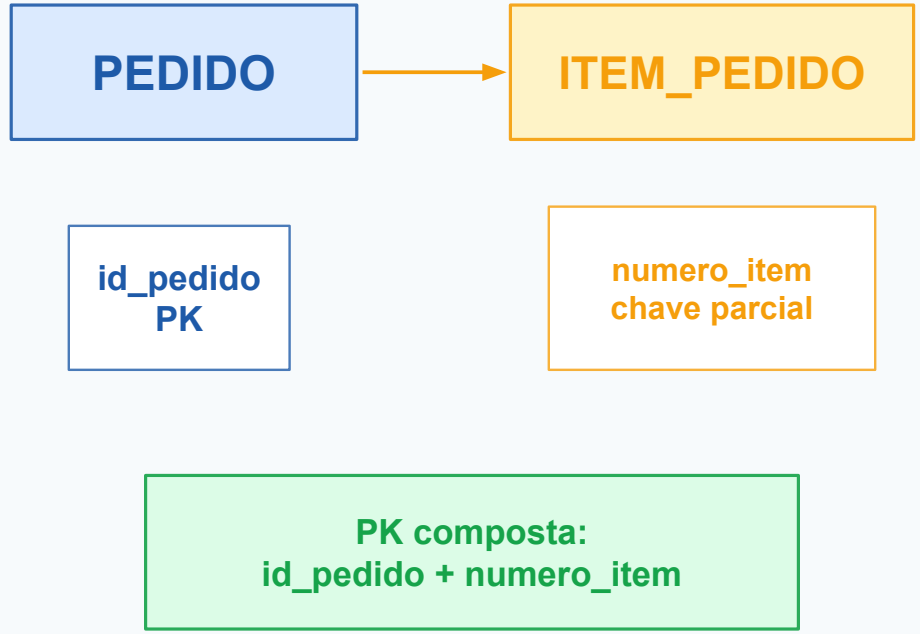


Complementa: Slide 05, entidades fortes/fracas e atributos-chave.

Exemplo: Pedido e ItemPedido

CHAVE COMPOSTA NA PRÁTICA

Considere Pedido(id_pedido, data) e ItemPedido(id_pedido, numero_item, produto, quantidade). O numero_item pode repetir em pedidos diferentes, mas a combinação id_pedido + numero_item identifica cada linha. Assim, a chave da entidade forte participa da identificação da fraca.

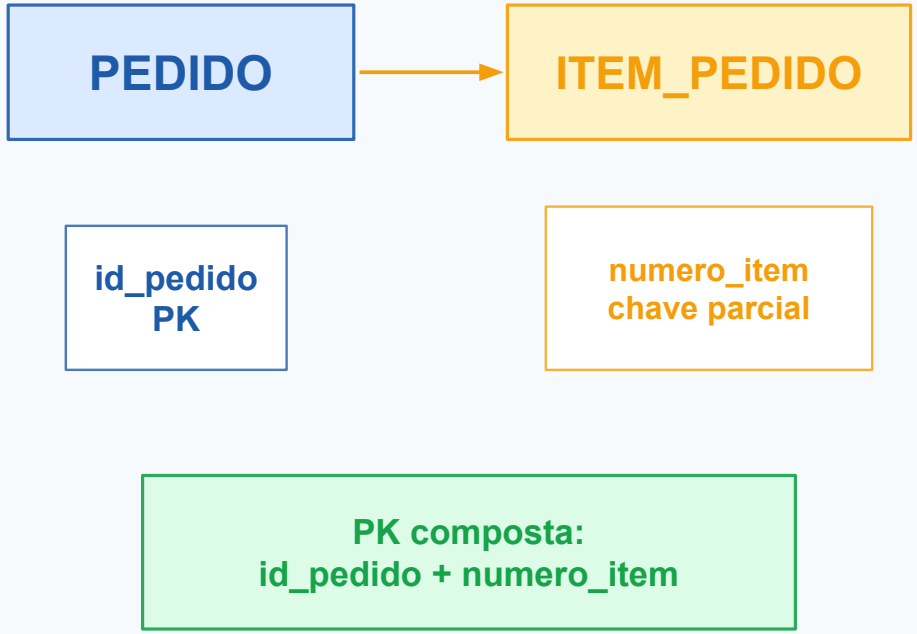


Aplicação prática: modelagem de vendas, notas fiscais, compras ou orçamentos.

Mapeando entidade fraca

DO DER PARA O MODELO RELACIONAL

Ao transformar uma entidade fraca em tabela, inclua a chave primária da entidade forte como chave estrangeira e também como parte da chave primária composta. Essa solução preserva a dependência de identificação e ajuda o SGBD a manter a integridade dos dados.



Revisar: Slide 04, chaves e integridade; Slide 06, chaves primária e estrangeira.

Prática: desenhar entidade fraca

USANDO DIAGRAMS.NET

No diagrams.net, crie a entidade Pedido, depois ItemPedido, ligue as duas por um relacionamento identificador e destaque numero_item como chave parcial. Em seguida, escreva o esquema relacional correspondente. Essa prática conecta diagrama, conceito e implementação.



Aplicação gratuita: diagrams.net — https://www.diagrams.net/

Quando um SGBD pode ser excesso?

CUSTO, COMPLEXIDADE E NECESSIDADE REAL

SGBDs oferecem segurança, integridade, controle de concorrência e consultas poderosas. Porém, também exigem instalação, manutenção, modelagem, backup e administração. Em problemas muito pequenos, locais e temporários, um arquivo simples pode ser suficiente e mais econômico.



Revisar: Slide 01, vantagens e desvantagens dos bancos de dados.

Situações para armazenamento simples

QUANDO ARQUIVO PODE RESOLVER

Arquivos CSV, JSON ou planilhas podem ser adequados quando os dados são pequenos, usados por uma única pessoa, têm baixa criticidade, não exigem muitas consultas simultâneas e podem ser recriados facilmente. Exemplos: lista temporária, configuração simples ou protótipo inicial.



Escolha cresce com:
risco, volume, usuários e regras

Ferramentas: planilhas, editores JSON, Python, SQLite para protótipos um pouco mais estruturados.

Quando o SGBD é recomendado

SINAIS DE QUE O PROBLEMA CRESCEU

Um SGBD é indicado quando há muitos usuários, dados importantes, necessidade de segurança, consultas variadas, regras de integridade, transações, backup confiável e crescimento esperado. Nesses casos, o custo de administrar o SGBD tende a ser menor que o risco de perder consistência.



Revisar: Slide 01, slides de vantagens; Slide 02, componentes do SGBD.

Checklist de decisão

PERGUNTAS ANTES DE ESCOLHER

Antes de escolher entre arquivo, SQLite ou SGBD servidor, pergunte: haverá acesso simultâneo? Os dados são críticos? Há regras de integridade? Será necessário auditar alterações? O volume crescerá? Existem relatórios variados? As respostas indicam o nível adequado de tecnologia.



Atividade: aplicar o checklist a uma agenda pessoal, biblioteca e sistema acadêmico.

Comparando alternativas

CSV, SQLITE E SGBD SERVIDOR

CSV e JSON são simples, mas têm pouco controle. SQLite é leve, funciona em arquivo único e é ótimo para estudo e aplicações locais. PostgreSQL, MySQL e SQL Server oferecem recursos robustos para múltiplos usuários, segurança, transações e administração profissional.



Escolha cresce com:
risco, volume, usuários e regras

Revisar: Slide 02, MySQL, PostgreSQL, Oracle e SQL Server.

Erros comuns de interpretação

O QUE EVITAR AO ESTUDAR

Evite decorar siglas sem entender finalidade, tratar usuário casual como usuário sem importância, confundir chave parcial com chave primária completa e afirmar que sempre se deve usar SGBD. Em Banco de Dados, quase sempre a resposta correta depende do contexto e das regras do problema.



Dica final: explique com exemplo sempre que aparecer um termo técnico.

Avaliação e referências

FIXAÇÃO FINAL E FONTES CONSULTADAS

Avaliação sugerida: 1) diferencie usuário casual e sofisticado; 2) classifique CREATE, SELECT e GRANT; 3) explique chave parcial em ItemPedido; 4) indique quando um arquivo simples basta; 5) justifique quando um SGBD é necessário. Referências: slides 01 a 06 da disciplina; PostgreSQL GRANT/REVOKE; IBM/SQL; Iowa Pressbooks sobre entidade fraca; DB Browser for SQLite; diagrams.net.

Links: postgresql.org/docs | sqlitebrowser.org | diagrams.net | pressbooks.uiowa.edu/fundamentalsofdatabasemanagement

